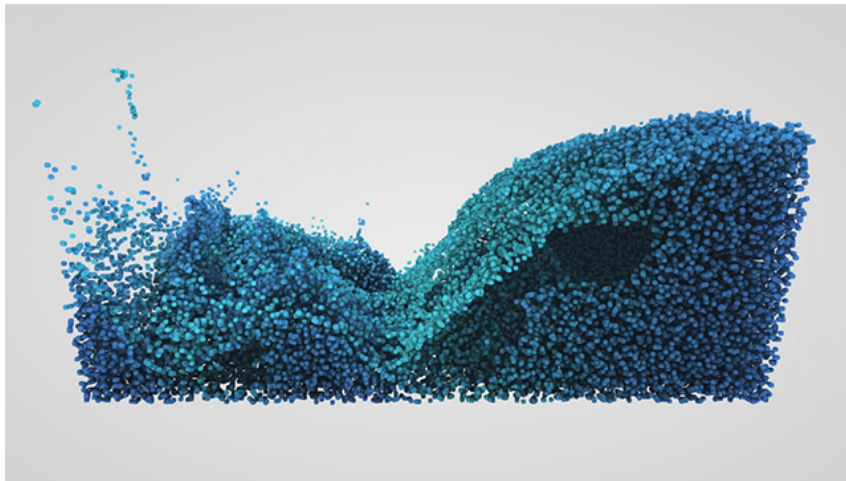


GPU Computing Project

Particle Simulation Acceleration using CUDA Programming



Project Report

Academic Year 2025–2026

Author

Julien Royer, Master of Artificial Intelligence

Supervisor

Christophe Picard, head of study Grenoble INP - Ensimag

June 9, 2026

Contents

1	Summary	3
2	Background and Challenges	4
2.1	The Verlet Integration Method	4
2.2	CPU Implementations	5
2.2.1	Simple Implementation	5
2.2.2	Refined Optimizations : Hash Grid Computation	6
2.2.3	Performance Comparison	7
2.3	Parallelization Opportunities	8
3	Simple CUDA Implementation	9
3.1	Overall GPU Architecture	9
3.2	Data Structures and Memory Layout	9
3.3	Kernels Implemented	10
3.3.1	Position Update and Boundary Collisions	10
3.3.2	Particle–Particle Collision Kernel	10
3.3.3	CUDA–OpenGL Interoperability	10
3.4	GPU Simulation Loop	10
3.5	Thread and Block Dimensions	11
3.6	Limitations of the Simple Implementation	11
4	Bottlenecks & Optimizations	12
4.1	Identification of the Main Bottleneck	12
4.2	Memory-Level Optimization: Shared Memory and Coalesced Accesses . . .	12
4.2.1	Motivation	12
4.2.2	Optimized Collision Kernel	12
4.3	Algorithmic Optimization: Spatial Hash Grid	13
4.3.1	GPU-Oriented Implementation	13
4.3.2	GPU-Specific Challenges	13
4.3.3	Design Trade-offs	13
5	Performance Evaluations	14
5.1	Benchmark Construction	14
5.1.1	Hardware Configuration	14
5.1.2	Measurement Pipeline	14
5.2	Execution Time Analysis	15
5.3	GPU Resource Utilization	16
5.4	Overall Performance Evaluation : CPU vs GPU	17

6	Conclusion	19
.1	Simulation Visualizations	20

Chapter 1

Summary

This project is part of the Master 2 High Performance and GPU Computing course, supervised by Christophe Picard, at Grenoble INP - Ensimag.

The aim of this project is to develop a GPU-accelerated 2D particle simulator capable of handling thousands to millions of interacting particles in real time, with the objective of simulating fluids. This project is based on a personal C++ particle simulator project using the Verlet integration method, implemented on a CPU. The goal of this project is to first identify parallelizable operations, then transform the code architecture to leverage the potential of GPU programming using CUDA, with the aim of drastically improving the simulator's performance. We will then compare CPU and GPU implementations, identify performance bottlenecks, and explore optimization techniques learned during the GPU computing course.

All the source code of the project, including the different implementation versions as well as the complete benchmarking pipeline, is available on GitHub at https://github.com/RoyerJu64/Particle_Simulation_GPU_Project.git.

Chapter 2

Background and Challenges

2.1 The Verlet Integration Method

Particle-based simulations are widely used to model various physical phenomena such as fluids, gases, granular materials, and plasmas. In this project, each particle follows Newtonian mechanics: for every time step, the total force acting on each particle is computed, then integrated to update its velocity and position. A basic physical model can be written as:

$$m_i \frac{d^2 \mathbf{x}_i}{dt^2} = \sum_{j \neq i} \mathbf{F}(\mathbf{x}_i, \mathbf{x}_j),$$

where $\mathbf{x}_i$ and m_i are the position and mass of particle i , and $\mathbf{F}$ represents the pairwise interaction force (e.g., gravitational, repulsive, or Lennard–Jones).

In this project, the time integration is performed using the **Verlet integration method**, which is widely used in molecular dynamics and particle simulations due to its simplicity. The standard form of the algorithm is:

$$\mathbf{x}_i(t + \Delta t) = 2\mathbf{x}_i(t) - \mathbf{x}_i(t - \Delta t) + \frac{\mathbf{F}_i(t)}{m_i} \Delta t^2.$$

Velocities can be estimated, if needed, as:

$$\mathbf{v}_i(t) = \frac{\mathbf{x}_i(t + \Delta t) - \mathbf{x}_i(t - \Delta t)}{2\Delta t}.$$

At each simulation time step, the core idea of the update loop is to first compute and update the new state of each particle, and then enforce physical constraints through **collision handling**. The particle velocity is implicitly evaluated using the **Verlet-based formulation** from the current and previous positions, ensuring numerical stability for the integration. Based on this velocity, **boundary collisions** with the domain walls and **particles-particles collisions** are detected and resolved by correcting the particle position and applying a **damping factor** to simulate energy loss during impacts. After collision handling, the new particle position is computed using the Verlet integration scheme, which combines the current position, velocity, and acceleration.

Algorithm 1 Particle Update with Verlet and Collisions

Require: currentPosition, previousPosition, acceleration, Δt

Ensure: updated currentPosition, previousPosition

1: Compute approximate velocity:

$$\mathbf{v} \leftarrow \frac{\mathbf{x}_{current} - \mathbf{x}_{previous}}{\Delta t}$$

2: Check boundary collisions and correct position:

 if $\mathbf{x}_{current}$ outside domain: correct position and invert $\mathbf{v}$ with damping

3: Check particle-particle collisions:

4: **for** each particle pair (i, j) **do**

5: **if** particles overlap **then**

6: Correct positions to remove overlap

7: Adjust velocities according to elastic or inelastic collision

8: **end if**

9: **end for**

10: Update position using Verlet:

$$\mathbf{x}_{new} \leftarrow 2\mathbf{x}_{current} - \mathbf{x}_{previous} + \mathbf{a}\Delta t^2$$

11: Store previous and current positions:

$$\mathbf{x}_{previous} \leftarrow \mathbf{x}_{current}, \quad \mathbf{x}_{current} \leftarrow \mathbf{x}_{new}$$

12: Reset acceleration (e.g., gravity)

2.2 CPU Implementations

2.2.1 Simple Implementation

In our Simple CPU-based implementation, each particle is represented as an **object** containing its current position, previous position, velocity, and acceleration. The numerical computations for the particle dynamics, including the integration of motion using the Verlet formula are performed entirely within a **dedicated solver class**.

For visualization, we use the **SFML library** (Simple and Fast Multimedia Library), which provides a lightweight interface for rendering 2D graphics and handling user input. SFML retrieves the computed particle states from the CPU and sends them to the GPU solely for display purposes; no physics or numerical calculations are performed on the GPU. Thus, all simulation computations remain **CPU-only**.

Our **algorithm 1** has a computational complexity of $O(n^2)$, where n is the number of particles (because of the handle collision loop), which makes it very demanding in terms of processing time and memory usage for large particle systems.

2.2.2 Refined Optimizations : Hash Grid Computation

A first simple improvement of our CPU implementation is the use of a **spatial hashing technique**, commonly referred to as a hash grid, to accelerate collision detection. Instead of checking every particle against all others, the simulation domain is divided into a **uniform grid of cells**, and each particle is assigned to a cell based on its position. Collisions are then only checked between particles within the same cell or in neighboring cells. This significantly reduces the number of pairwise checks while preserving accuracy, providing a simple yet **effective way to improve the performance** of the collision detection step without modifying the core solver logic.

Algorithm 2 Particle Update with Verlet Integration and Hash Grid Collision Detection

Require: currentPosition, previousPosition, acceleration, Δt , gridSize

Ensure: updated currentPosition, previousPosition

1: Compute new position using Verlet:

$$\mathbf{x}_{new} \leftarrow 2\mathbf{x}_{current} - \mathbf{x}_{previous} + \mathbf{a}_{current}\Delta t^2$$

2: Check boundary collisions:

3: **if** particle outside domain **then**

4: Correct $\mathbf{x}_{new}$

5: Apply damping to velocity if needed:

$$\mathbf{v} \leftarrow \frac{\mathbf{x}_{new} - \mathbf{x}_{current}}{\Delta t}$$

6: **end if**

7: Assign each particle to a cell of the hash grid based on position:

$$cell_index \leftarrow \text{hash}(\mathbf{x}_{new}, gridSize)$$

8: Check particle–particle collisions using hash grid:

9: **for** each particle i **do**

10: **for** each particle j in the same or neighboring cells of i **do**

11: **if** particles i and j overlap **then**

12: Correct positions to remove overlap

13: Adjust velocities according to elastic/inelastic rules

14: **end if**

15: **end for**

16: **end for**

17: Update positions for next step:

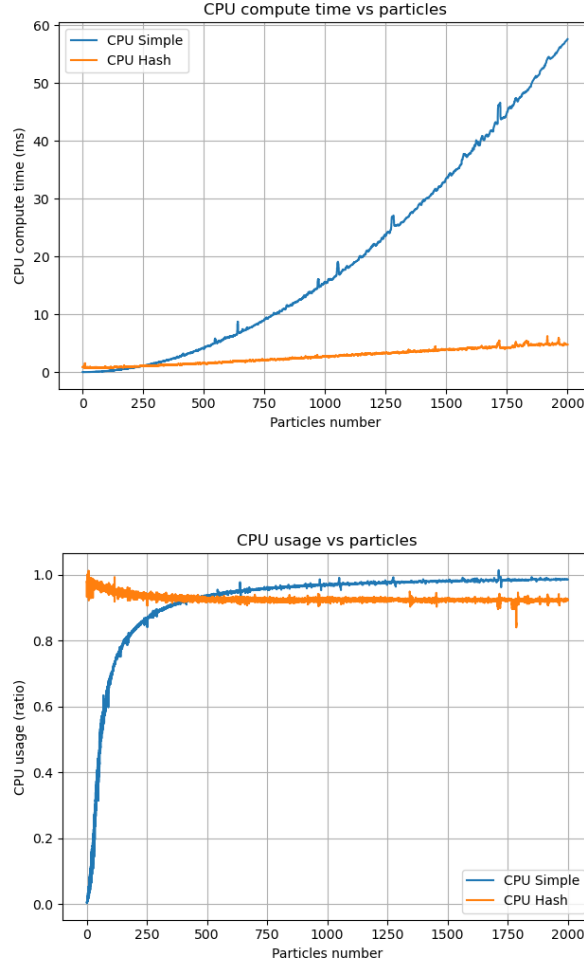
$$\mathbf{x}_{previous} \leftarrow \mathbf{x}_{current}, \quad \mathbf{x}_{current} \leftarrow \mathbf{x}_{new}$$

18: Reset acceleration if necessary (e.g., gravity)

By using the hash grid, the number of collision checks is greatly reduced, lowering the average computational complexity from $O(n^2)$ to approximately $O(n)$ for uniformly distributed particles.

2.2.3 Performance Comparison

We can evaluate the performances of these two CPU-only implementations by collecting some data on the execution time and the on the CPU utilization while running.



The two plots clearly highlight the impact of **algorithmic complexity** on the performance of the **CPU-only implementations**. On the first plot, representing the **CPU compute time** as a function of the **number of particles**, the **simple approach** exhibits a **rapidly increasing curve**, consistent with a **quadratic $O(n^2)$ complexity**. In contrast, the **hash grid implementation** shows a much **slower, near-linear growth**, confirming the **effectiveness of spatial partitioning** in reducing the **number of collision evaluations**. This difference becomes increasingly **pronounced as the number of particles grows**.

The second plot, showing **CPU usage**, further supports this observation: while the **simple implementation** quickly **saturates the CPU** as the particle count increases, the **hash grid version** maintains a more **stable and slightly lower CPU utilization**.

Overall, these results show that even with **algorithmic improvements**, **CPU-only implementations** quickly become **impractical** for relatively **small numbers of particles**, making it **necessary to parallelize the computations on the GPU** to achieve **real-time performance**.

2.3 Parallelization Opportunities

Our numerical scheme is particularly well-suited for GPU execution, as the update of each particle depends only on its own state and on locally evaluated interaction forces, enabling a high degree of **fine-grained data parallelism**. On the GPU, each particle can be mapped to a single CUDA thread, allowing thousands of particle updates to be computed simultaneously. This mapping naturally maximizes occupancy when the number of particles is large, which is what we want.

The inner interaction loop remains the main computational bottleneck and therefore the primary target for GPU acceleration. Several optimization strategies can be explored to further improve performance:

- **Parallel particle updates:** particle integration (position, velocity, and acceleration updates) can be executed independently for each particle, making it ideal for massive parallel execution on the GPU.
- **Memory access optimization:** particle data can be stored in a structure-of-arrays (SoA) layout to ensure coalesced global memory accesses, significantly improving memory throughput compared to an array-of-structures (AoS) layout.
- **Shared memory usage:** for interaction computations, blocks of neighboring particle data can be loaded into shared memory, reducing global memory accesses and hiding memory latency during force accumulation.
- **Computation–rendering interoperability:** the CPU implementation relies on the SFML graphical library for visualization. On the GPU, particle positions can be stored directly in an OpenGL Vertex Buffer Object (VBO) and updated using CUDA–OpenGL interoperability, eliminating costly CPU–GPU memory transfers and enabling real-time rendering directly from GPU-computed data.

Chapter 3

Simple CUDA Implementation

We will now present a first GPU-based implementation of the particle simulation using CUDA. The objective is not yet to achieve optimal performance, but rather to demonstrate how the original CPU algorithm can be mapped onto the GPU execution model in a clear and efficient way. This simple implementation also serves as a baseline for the more advanced optimizations introduced in the next chapter.

3.1 Overall GPU Architecture

The simulation is **fully executed on the GPU**, while the CPU is only responsible for initialization and kernel launches. The implementation is divided into two main components: a **GPUSolver**, which handles the **physical simulation**, and a **GPURenderer**, which manages **visualization** through **OpenGL**.

Each CUDA thread is associated with a single particle. At each time step, the GPU performs the following operations:

1. update particle positions using the Verlet integration scheme,
2. resolve collisions with the simulation boundaries,
3. handle particle–particle collisions,
4. write updated positions directly into an OpenGL vertex buffer for rendering.

This design allows the entire simulation loop to remain on the GPU, **minimizing data transfers between the CPU and the GPU**.

3.2 Data Structures and Memory Layout

Particles are stored in a **contiguous array** allocated in **global GPU memory**. Each particle is represented by its **current position**, **previous position**, **velocity** and **acceleration**.

Several **physical parameters** remain constant throughout the simulation, such as the **time step**, **particle radius**, **gravity**, and the **simulation bounds**. These parameters are stored in CUDA **constant memory**.

Since **constant memory** is **cached** and optimized for **broadcast access**, this choice significantly reduces **memory access latency** when the same values are read by all threads.

3.3 Kernels Implemented

3.3.1 Position Update and Boundary Collisions

The first CUDA kernel updates particle positions using a **Verlet integration scheme**. Each thread operates on a **single particle**, making this kernel **embarrassingly parallel**.

The position update is given by:

$$\mathbf{x}_{t+\Delta t} = 2\mathbf{x}_t - \mathbf{x}_{t-\Delta t} + \mathbf{a}_t\Delta t^2,$$

Boundary collisions with the simulation domain are handled within the **same kernel**. When a particle exits the allowed domain, its position is **clamped** and its velocity is reflected with a **damping** factor. Grouping **integration** and **boundary handling** in a single kernel reduces **kernel launch overhead** and avoids **unnecessary memory accesses**.

3.3.2 Particle–Particle Collision Kernel

Particle–particle interactions are handled by a **second CUDA kernel**. Each thread is responsible for resolving collisions involving **one particle**, looping over **all other particles** to detect overlaps.

This approach leads to a **quadratic computational complexity** of $\mathcal{O}(N^2)$, which quickly becomes the **main performance bottleneck** as the number of particles increases. Nevertheless, this **naive strategy** is sufficient for a **first GPU implementation** and allows a **direct comparison** with the CPU version.

3.3.3 CUDA–OpenGL Interoperability

To render the simulation efficiently, particle positions are written directly into an **OpenGL Vertex Buffer Object (VBO)**. The buffer is registered with CUDA using **CUDA–OpenGL interoperability**, allowing CUDA kernels to **access it directly**.

This approach avoids **costly CPU–GPU memory transfers** and ensures that rendering consumes the **most up-to-date particle positions**. The visualization pipeline therefore remains **entirely on the GPU**, significantly improving **performance** and **scalability**.

3.4 GPU Simulation Loop

At each simulation step, the following sequence of **CUDA kernels** is executed:

1. **position update and boundary collision kernel**,
2. **particle–particle collision kernel**,
3. **kernel writing particle positions to the OpenGL VBO**.

Since the application is written in C++, **CUDA kernels** cannot be called directly from the **host code**. A dedicated C++ **wrapper** is therefore used to **launch the kernels**, **manage execution configuration** (grid and block sizes), and **handle synchronization**. This wrapper provides a **clean interface** between the **CPU-side simulation loop** and the **GPU-side computation** while preserving **code modularity** and **readability**.

Algorithm 3 GPU simulation update loop

Require: Particle array on GPU, number of particles N

Ensure: Updated particle positions and VBO ready for rendering

```
1: if  $N = 0$  then
2:   return
3: end if
4: Compute grid and block dimensions
5: Launch kernel: update positions and boundary collisions
6:   updatePositionsKernel«<grid, block>>(particles,  $N$ )
7: Launch kernel: particle–particle collisions
8:   handleCollisionsKernel«<grid, block>>(particles,  $N$ )
9: Synchronize GPU execution
10: Map OpenGL VBO for CUDA access
11: Launch kernel: copy particle positions to VBO
12:   copyToVBOKernel«<grid, block>>(particles, vbo,  $N$ )
13: Unmap OpenGL VBO
```

3.5 Thread and Block Dimensions

The CUDA kernels are launched using a one-dimensional grid configuration, where each thread is responsible for processing a single particle.

$$\text{grid.x} = \left\lceil \frac{N_{\text{particles}}}{\text{blockSize.x}} \right\rceil \quad (3.1)$$

This ensures that all particles are covered, even when the particle count is not a multiple of the block size.

A fixed block size (`blockSize.x`) is used for all kernels, providing a consistent execution configuration across the different computation stages (position update, collision handling, and VBO transfer). In practice, a block size of **256 threads** was selected, which is a well-established compromise on modern NVIDIA GPUs.

All kernels therefore share the same launch configuration: `updatePositionsKernel`, `handleCollisionsKernel`, and `copyToVBOKernel`.

3.6 Limitations of the Simple Implementation

While this CUDA implementation demonstrates the feasibility of executing the simulation entirely on the GPU, it suffers from several limitations:

- the particle–particle collision kernel scales quadratically with the number of particles,
- memory access patterns are not optimized for coalescing,
- shared memory is not yet exploited,
- potential write conflicts are not explicitly handled.

These limitations highlight the necessity of more advanced GPU-oriented optimizations, which are introduced and evaluated in the next chapter.

Chapter 4

Bottlenecks & Optimizations

We will now try to improve the performance of the initial GPU implementation and presents two successive optimization strategies. Each optimization aims at improving scalability and computational efficiency while highlighting the practical limitations identified in the previous part.

4.1 Identification of the Main Bottleneck

Although our program now exhibits a high degree of parallelism, its performance is limited by:

- excessive **global memory accesses**,
- poor data reuse across threads.

These observations motivated the introduction of memory-level and algorithmic optimizations.

4.2 Memory-Level Optimization: Shared Memory and Coalesced Accesses

4.2.1 Motivation

The first optimization focuses on reducing **global memory traffic** by improving data locality. Since multiple threads repeatedly access neighboring particle data, caching this information in **shared memory** can significantly reduce memory latency.

4.2.2 Optimized Collision Kernel

Particles are processed in **tiles**, where each thread block loads a subset of particles into shared memory. Each thread then computes collisions between its assigned particle and the particles stored in shared memory.

This approach offers several advantages. While the asymptotic complexity remains $\mathcal{O}(N^2)$, the constant factor is significantly reduced, leading to noticeable performance gains for moderate particle counts.

Despite these improvements, the quadratic nature of the algorithm remains the primary scalability limitation.

4.3 Algorithmic Optimization: Spatial Hash Grid

4.3.1 GPU-Oriented Implementation

On the GPU, the spatial hash grid must be constructed and queried in a fully parallel manner. Each particle independently computes its **cell index** from its position and writes this information to a dedicated array.

Particles are then **sorted by cell index**, allowing particles belonging to the same cell to be stored contiguously in memory. This layout enables efficient range-based access, where each cell is represented by a start and end index in the sorted particle array.

Collision detection is performed by having each thread iterate only over particles contained in the neighboring cells of its own cell.

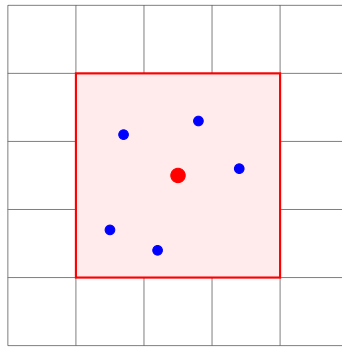


Figure 4.1: spatial hash grid representation.

4.3.2 GPU-Specific Challenges

While the spatial hash grid effectively reduces computational complexity, its GPU implementation introduces several non-trivial challenges.

First, collision resolution requires **concurrent updates** to particle positions. When multiple threads attempt to correct overlaps involving the same particle, **race conditions** naturally arise. Second, attempts to combine the hash grid with **shared memory** optimizations led to **non-physical behaviors**, including:

- asymmetric collision responses,
- artificial energy gain or dissipation,
- particle clustering and numerical jittering.

These artifacts stem from inconsistent update ordering and the lack of a global synchronization mechanism within a single kernel launch.

4.3.3 Design Trade-offs

To preserve physical stability and determinism, the final implementation with the Hash Grid prioritizes **correctness over raw performance**. Collision resolution is therefore performed using carefully controlled updates in **global memory**, avoiding aggressive shared memory usage. These experiments highlight a key trade-off in GPU programming between **performance**, **algorithmic complexity**, and **numerical stability**.

Chapter 5

Performance Evaluations

It is important now to present a detailed **performance analysis** of the proposed **particle simulation implementations**. The objective is to quantitatively evaluate the impact of **GPU acceleration** and the different **optimization strategies** introduced in the previous chapters.

To this end, a dedicated **performance measurement pipeline** was designed and implemented. It enables the systematic collection of **execution times**, as well as **memory** and **computational utilization metrics** on **GPU**.

The collected **data** are then analyzed through a series of **plots**, allowing direct comparison between **GPU-based implementations**.

5.1 Benchmark Construction

5.1.1 Hardware Configuration

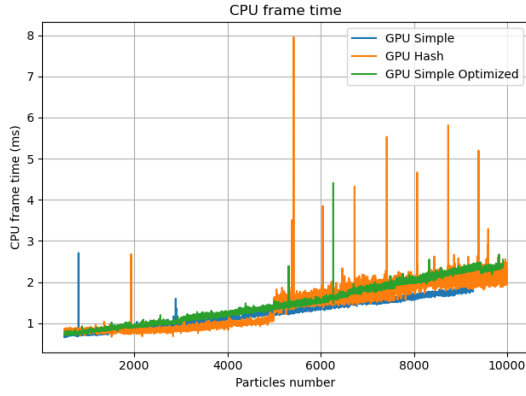
All benchmarks were conducted on the following system:

- **CPU:** 13th Gen Intel i9-13900H (20 threads)
- **GPU:** NVIDIA GeForce RTX 4060 Max-Q
 - **CUDA Cores:** 3072
 - **Memory:** 8 GB GDDR6, 128-bit bus
 - **TDP:** 80 W (Max-Q power-efficient design)
- **System Memory:** 31.8 GB

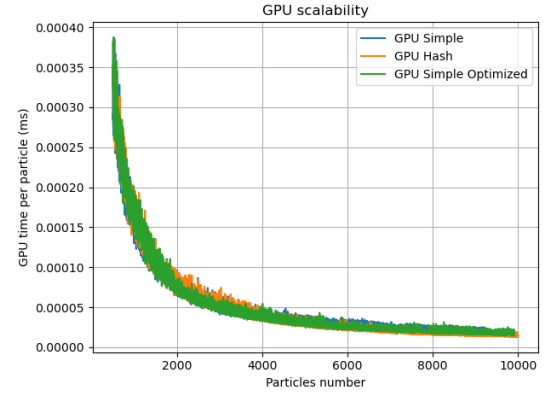
5.1.2 Measurement Pipeline

Benchmarks were performed by running the GPU particle simulation programs while continuously recording GPU utilization, memory usage, and power draw using `nvidia-smi`. GPU update times were logged directly by the application into CSV files for later analysis.

5.2 Execution Time Analysis



(a) CPU Time



(b) GPU Time

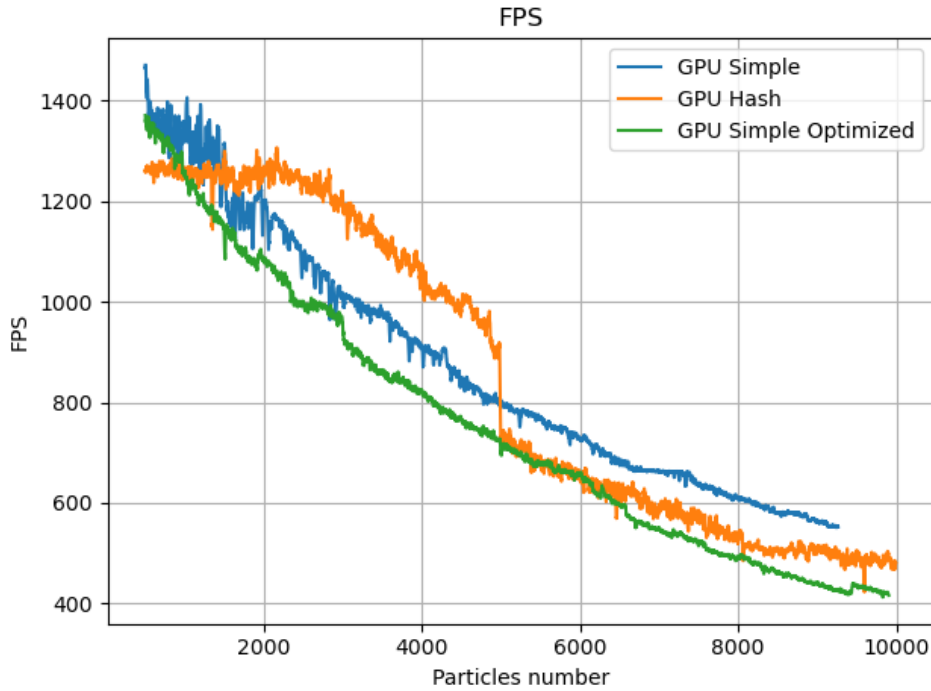


Figure 5.2: (c) FPS

The **performance results** presented in those figures highlight clear differences between the three **GPU-based implementations** as the number of particles increases.

The **GPU_Simple** implementation, which consists of the direct port of the CPU algorithm to the GPU without major optimizations, exhibits a **smooth and stable increase in CPU frame time** as particle count grows, reflecting the expected **linear scaling** of the workload.

In contrast, the **GPU_Hash** implementation, which introduces a **spatial hash grid** for collision handling, shows **high variability in CPU frame time**, with frequent spikes appearing beyond approximately 5,000 **particles**. This behavior could be caused by irregular memory accesses and increased synchronization overhead, leading to a **sharp FPS drop** in dense particle regimes.

Finally, the **GPU scalability plot** highlights the benefits of GPU parallelism. For all implementations, the per-particle cost decreases as the number of particles increases, confirming effective GPU utilization. However, **GPU_Optimized** achieves the **lowest GPU time per particle**, especially in the high-particle regime, indicating better memory efficiency and occupancy.

5.3 GPU Resource Utilization

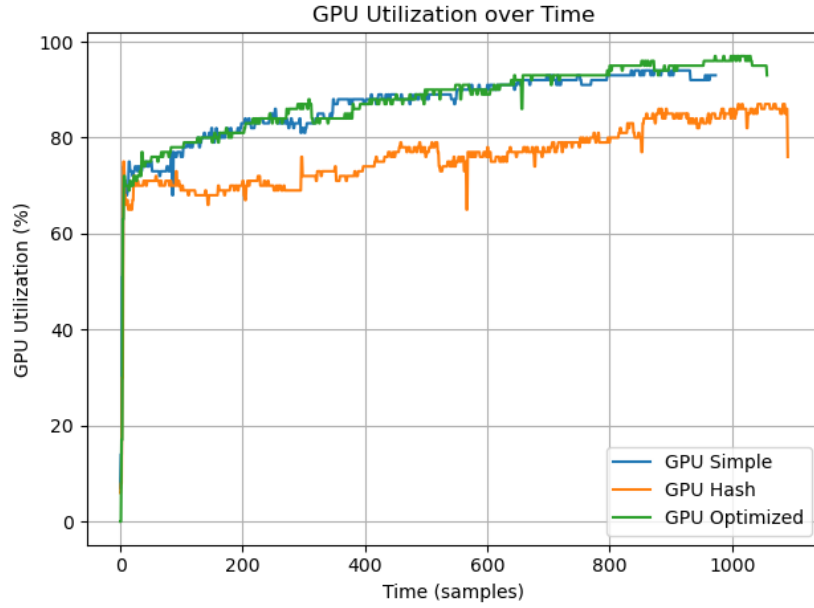


Figure 5.3: GPU Compute Utilization

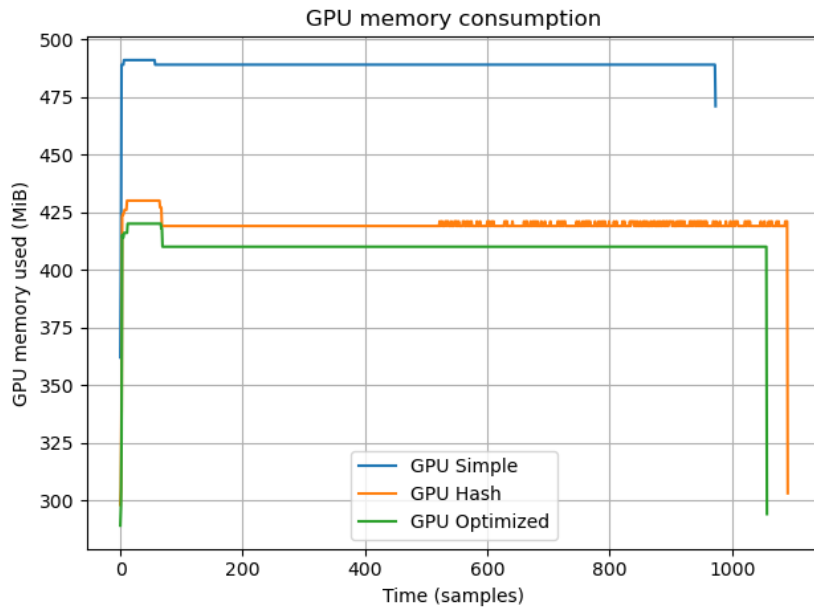
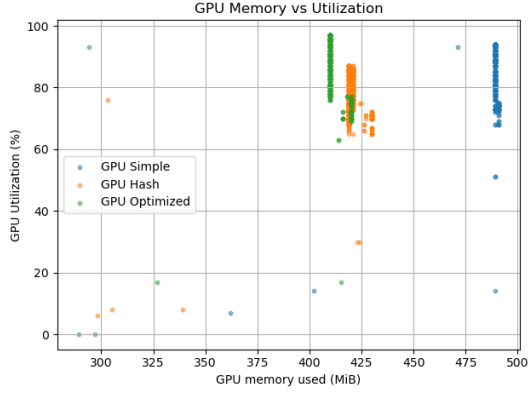
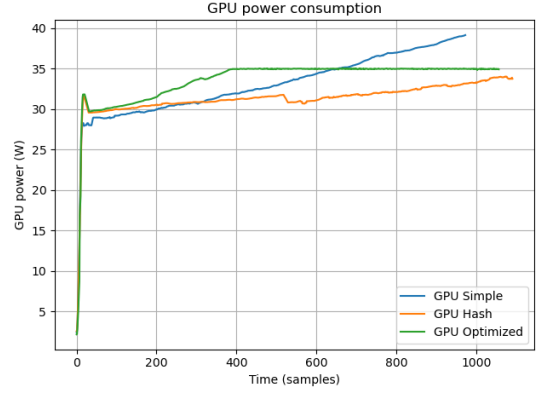


Figure 5.4: Memory Utilization



(a)



(b)

These focuses on the GPU resources data reveal **significant performance differences** across our different GPU implementation strategies for particle simulations.

The **GPU_Simple** implementation exhibits consistently high memory consumption at approximately **490 MiB** throughout execution, while achieving near-complete **GPU utilization (95–98%)** after an initial ramp-up period.

The **GPU_Hash** implementation demonstrates a substantial reduction in memory usage to roughly **420 MiB**, corresponding to a **14% improvement** over the simple approach. This gain comes with reduced GPU utilization, fluctuating between **65–85%** with noticeable volatility. This behavior indicates irregular memory access patterns and potential thread divergence introduced by the hash grid.

Most notably, the **GPU_Optimized** implementation achieves the **best overall balance** by maintaining memory consumption in the **410–420 MiB** range while sustaining high GPU utilization between **85–95%**. The resulting utilization is both high and stable, indicating the successful removal of memory bottlenecks without introducing thread divergence. The memory–utilization scatter plot shows tight clustering with consistently high utilization, highlighting the **superior stability** of this approach.

5.4 Overall Performance Evaluation : CPU vs GPU

Table 5.1: CPU benchmark summary

Metric	CPU Simple	CPU Hash
Maximum number of particles	2000	2000
Maximum CPU time (ms)	33.370	8.245
Average CPU time (ms)	1.135	2.010
Minimum FPS	29.5	110.0
Average CPU usage (%)	57.4	93.2

Table 5.2: GPU profiling and performance summary

Metric	GPU Simple	GPU Hash	GPU Optimized
Average GPU utilization (%)	86.6	76.2	82.7
Maximum GPU utilization (%)	94.0	87.0	93.0
Maximum memory usage (MiB)	491	430	519
Average power draw (W)	33.0	31.3	37.1
Mean GPU time (ms)	0.1687	0.1507	0.1482
GPU time standard deviation (ms)	0.0186	0.0170	0.0200
Mean FPS	873.5	852.4	825.6
Minimum FPS	527.9	303.8	423.6

The results in Tables 5.1 and 5.2 clearly highlight the **dramatic performance gap between CPU and GPU executions**. Even with algorithmic improvements such as spatial hashing, the CPU implementations remain fundamentally limited by sequential execution and rapidly reach high utilization levels. While the `CPU_Hash` approach improves scalability compared to the naive CPU version, it does so at the cost of **near-saturation of CPU resources** and still delivers frame rates an order of magnitude lower than GPU-based solutions.

In contrast, all GPU implementations achieve **sub-millisecond execution times** and sustain average frame rates above **800 FPS**, confirming the effectiveness of massively parallel execution for particle simulations. The naive GPU port already provides a speedup of more than one order of magnitude compared to the best CPU configuration, confirming that the computational workload is highly GPU-friendly.

Most importantly, the optimized GPU implementation achieves the **lowest mean GPU execution time** while maintaining high utilization and stable performance. Overall, these results validate that both the **GPU porting effort** and the applied **low-level optimizations** are highly effective, transforming a CPU-bound simulation into a high-performance GPU solution.

Chapter 6

Conclusion

As a conclusion, this project investigated the design and optimization of a GPU-accelerated program, with a particular focus on performance analysis and architectural trade-offs. Through successive implementations, ranging from a direct GPU port to highly optimized kernels, the study highlighted the critical impact of memory access patterns, parallel efficiency, and algorithmic choices on overall performance. Beyond the technical results, this project proved to be highly enriching, providing valuable hands-on experience in GPU programming, performance profiling, and optimization strategies. All source code, implementation variants, and benchmarking tools are publicly available on GitHub at https://github.com/RoyerJu64/Particle_Simulation_GPU_Project.git.

.1 Simulation Visualizations

This appendix provides screenshots of the particle simulations. These visualizations illustrate the dynamic behavior of the particle system.

